



Gyan Ganga Institute of Technology and Sciences

Department of Computer Science and Engineering
Branch: Data Science

Project Synopsis on **Prahari : ML-Based Banking APK** **Classifier & Risk Assessment System**

In partial fulfillment for the award of the degree
of
BACHELOR OF ENGINEERING
in
Computer Science and Engineering

Submitted By:

Vivek Kumar Barman
Piyush Kumar Patel

0206CD231154
0206CD231090

5th Semester
2023-2027



Gyan Ganga Institute of Technology and Sciences

Department of Computer Science and Engineering
Branch: Data Science

Project Synopsis on **Prahari : ML-Based Banking APK** **Classifier & Risk Assessment System**

In partial fulfillment for the award of the degree
of
BACHELOR OF TECHNOLOGY
in
Computer Science and Engineering - Data Science

Submitted By:

Vivek Kumar Barman
Piyush Kumar Patel

0206CD231154
0206CD231090

5th Semester
2023-2027



Index

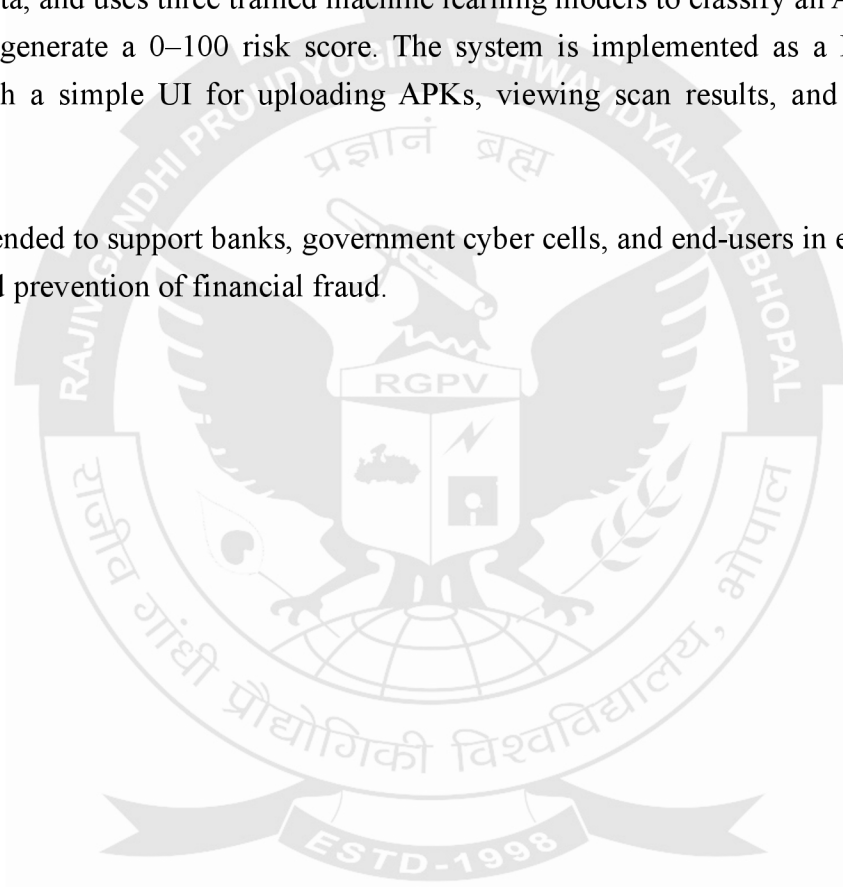
Title	Page No.
• Abstract	
• Chapter 1:- Objective/Introduction -----	1-2
• Chapter 2:- Usefulness and Innovation -----	3-5
• Chapter 3:- System overview & Methodology -----	6-8
• Chapter 4:- Applications -----	9
• Chapter 5:- Future scope -----	10
• Chapter 6:- Conclusion -----	11
• Bibliography -----	12

ABSTRACT

Fake banking applications are commonly used in financial scams to steal user credentials, OTPs, and sensitive banking information. Attackers distribute modified or completely fake APKs that visually mimic legitimate banking apps. Traditional signature-based antivirus tools and generic APK analyzers are not optimized for such banking-specific threats.

Prahari is an ML-based static analysis system that focuses specifically on banking APKs. It extracts features such as package name patterns, permission combinations, certificate properties, and file metadata, and uses three trained machine learning models to classify an APK as benign or malicious and generate a 0–100 risk score. The system is implemented as a Flask-based web application with a simple UI for uploading APKs, viewing scan results, and generating PDF reports.

This tool is intended to support banks, government cyber cells, and end-users in early detection of scam APKs and prevention of financial fraud.



CHAPTER 1: INTRODUCTION

Mobile banking and UPI-based apps have become a critical part of day-to-day financial transactions. Along with this growth, cybercriminals have increasingly started using fake Android banking applications for fraud.

These APKs may:

- Copy the name and icon of genuine banking apps
- Request dangerous permissions such as SMS, CALL, CONTACTS, STORAGE
- Use untrusted or suspicious certificates
- Send collected information to attacker-controlled servers

Many users download APKs from links shared over WhatsApp / SMS / social engineering calls rather than from official app stores, making them highly vulnerable.

1.1 Problem Definition

There is a lack of domain-specific tools that can automatically analyze a banking APK and tell whether it looks legitimate or suspicious based on:

- Package identity
- Permission usage patterns
- Signing certificate properties

Most existing tools are either generic malware detectors or manual static analysis tools that require expertise.

1.2 Objectives

The main objectives of Prahari are:

- To analyze banking-related APKs statically without installing them
- To use machine learning models trained on a banking-specific dataset
- To classify each APK as benign or malicious and compute a risk score
- To present results through a simple web interface suitable for non-experts
- To store scan history in a database for later reference and analysis

1.3 Scope

Prahari is designed as a Minor Project MVP that:

- Works on one APK at a time
- Focuses mainly on banking and finance APKs
- Uses static analysis only (no runtime behavior currently)
- Operates as an offline tool (no dependency on external cloud services)

1.4 Ethical & Legal Considerations

- No confidential APK data is stored
- Datasets are synthetically generated only
- Tool assists in scam prevention, not hacking
- Only static analysis is performed; no runtime execution
- User privacy & data protection ensured



CHAPTER 2: USEFULNESS & INNOVATION

2.1 Usefulness

Prahari can be used by:

- Bank security teams – to validate whether a distributed APK is legitimate
- Government / CERT / Cybercrime units – to triage suspicious apps reported by citizens
- General users – to verify an APK before installing it
- Researchers and students – to study feature patterns in benign vs malicious banking apps

It helps in:

- Early detection of fake apps used in fraud
- Identifying risky permissions and untrusted certificates
- Reducing reliance on manual static reverse engineering

2.2 Innovation

What makes Prahari different from a normal APK analyzer:

1. Domain-Specific ML

- Models are trained on 50,000+ synthetic but domain-guided samples for each feature type:
 - packages_train.csv
 - permissions_train.csv
 - certificates_train.csv
- Data patterns are derived from real banking APKs but generated synthetically to avoid legal and privacy issues.

2. Three Independent ML Models (Ensemble Logic)

- PackageNameDetector (XGBoost)
 - Uses package name, app name, app size and SDK levels
 - Realistic accuracy: ~92.4%
- PermissionPatternDetector (Logistic Regression)
 - Learns dangerous permission combinations typical in trojans/droppers
 - Realistic accuracy: ~87.3%
- CertificateDetector (XGBoost)
 - Uses certificate subject, issuer, validity period, and key size
 - Realistic accuracy: ~89.6%

3. Risk Score (0–100) with ML Confidence

- Combines package, permissions, and certificate risk into a single numeric score, making it easy to interpret.

4. Banking-Focused Synthetic Dataset Generation

- You wrote custom generator scripts for:
 - certificates (certificate_subject / issuer / validity / key size)
 - packages (bank-style vs suspicious style package names, sizes, SDKs)
 - permissions (banking patterns vs malware droppers)

2.3 System Description

The system is divided into four major modules:

- User Interface – Upload APK, view risk score, view report
- Static Analysis Engine – Extract package metadata, certificate info, permissions
- Machine Learning Engine – Predict suspiciousness using three trained models
- Database & Reporting – Store scan results and generate detailed reports

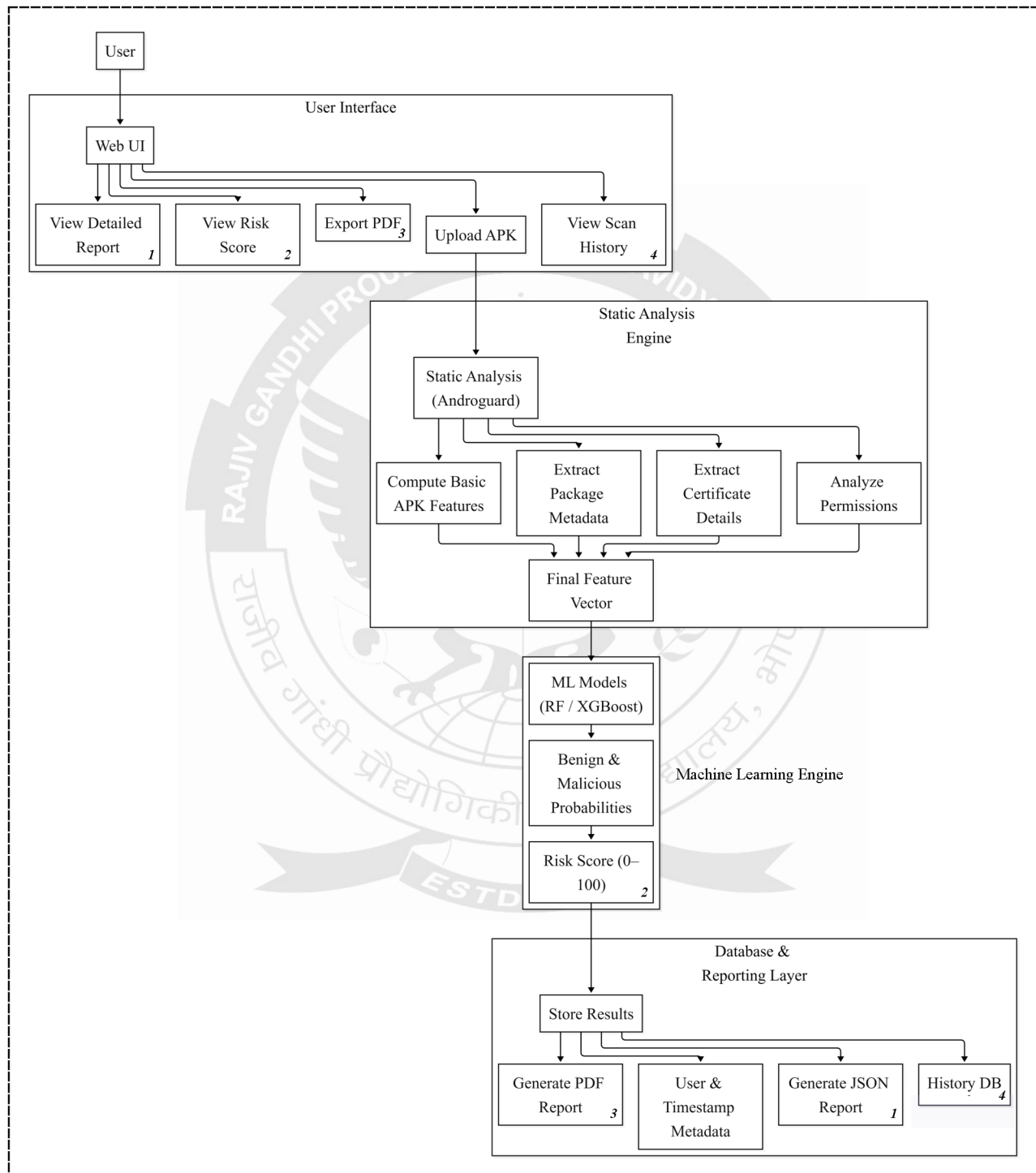


Figure 3.1 – System Architecture Diagram

2.4 Backend Web Interface & Database Design

- Flask Web UI – Upload → Analyze → View report
- SQLite DB – Stores scan results & timestamps
- Templates – Single-page dashboard

Database Schema:

```
scans(  
  id INTEGER PRIMARY KEY,  
  file_info TEXT,  
  apk_metadata TEXT,  
  certificate_info TEXT,  
  permissions TEXT,  
  flags TEXT,  
  ml_prediction_result TEXT,  
  analysis_timestamp TEXT  
)
```



CHAPTER 3: SYSTEM OVERVIEW & METHODOLOGY

3.1 Workflow / Process Flow

Below steps are executed for every APK:

1. The user uploads the APK file through the web interface.
2. The APK is temporarily stored and validated.
3. Static analysis is performed using Androguard.
4. Feature extraction is carried out, including:
 - a. Package name, SDK version, and file size
 - b. Permissions requested by the application
 - c. Certificate and signing details
5. Each machine learning model predicts:
 - a. Benign probability
 - b. Malicious probability
6. A weighted risk score (0–100) is calculated.
7. A detailed JSON result is generated and stored in the database.
8. The UI displays the risk level and detailed report.
9. The user can export the report as a PDF or view the history of scanned APKs.

3.2 Techniques & Algorithms Used

Module	Technique	Purpose
Package Name Analysis	XGBoost Classifier	Detect spoofed naming conventions & abnormal metadata
Permission Analysis	Logistic Regression	Spot risky permission combinations typical in droppers
Certificate Analysis	XGBoost Classifier	Validate authenticity of signing certificate
Feature Vectorization	TF-IDF & DictVectorizer	Convert text & structured fields into ML-readable format
Risk Scoring	Weighted confidence ratio	Interpretability of prediction

- XGBoost performs well with mixed numeric + text features
- Logistic Regression works well for sparse permission vectors
- Ensemble → higher reliability than single classifier

3.3 Data Processing & Feature Engineering

Prahari uses static analysis only — the APK is never executed.

Feature Group	Extracted Features
Package Info	package_name, app_name, min_sdk, target_sdk, app size
Permissions	total count, combinations, frequency of risky permissions
Certificate Data	subject CN, issuer CN, key size, validity period, expiry

All features are normalized and transformed before ML.

3.4 ML Training Pipeline

Based on custom synthetic datasets (50K records per model)

Pipeline Steps:

1. Pattern extraction from real banking apps
2. Synthetic dataset generation (3 scripts)
3. Data split → 80% train / 20% test
4. Feature preprocessing (TF-IDF, scaling etc.)
5. Model training & hyperparameter tuning
6. Model validation with test data
7. Export .pkl models for real-time prediction

Model	Accuracy
Package Model	92.4%
Certificate Model	89.6%
Permission Model	87.3%

Output includes:

- Risk Score = (Pkg * 0.35) + (Perm * 0.35) + (Cert * 0.30)
- ML Confidence
- MD5 + SHA-256 fingerprints
- All the other information parsed in a form of compliant report

3.6 Tools & Technologies Used

Category	Technology
Frontend	HTML, CSS (Tailwind), JavaScript
Backend	Python + Flask
Static Analysis	Androguard
ML Libraries	Scikit-Learn, XGBoost
Cryptography	PyCryptodome / Cryptography
Database	SQLite3
Reporting	HTML → PDF export
Version Control	GitHub

Environment:

- OS: Windows / Linux
- Python 3.10+

3.7 Implementation Plan

Phase	Module
Phase-1	APK Static Parser
Phase-2	Dataset Generation
Phase-3	ML Model Training
Phase-4	Flask Web Integration
Phase-5	PDF Report + DB Logging

Deployment:

- Local web server → Browser-based UI → Offline scanning

CHAPTER 4: APPLICATIONS

Prahari can be applied in:

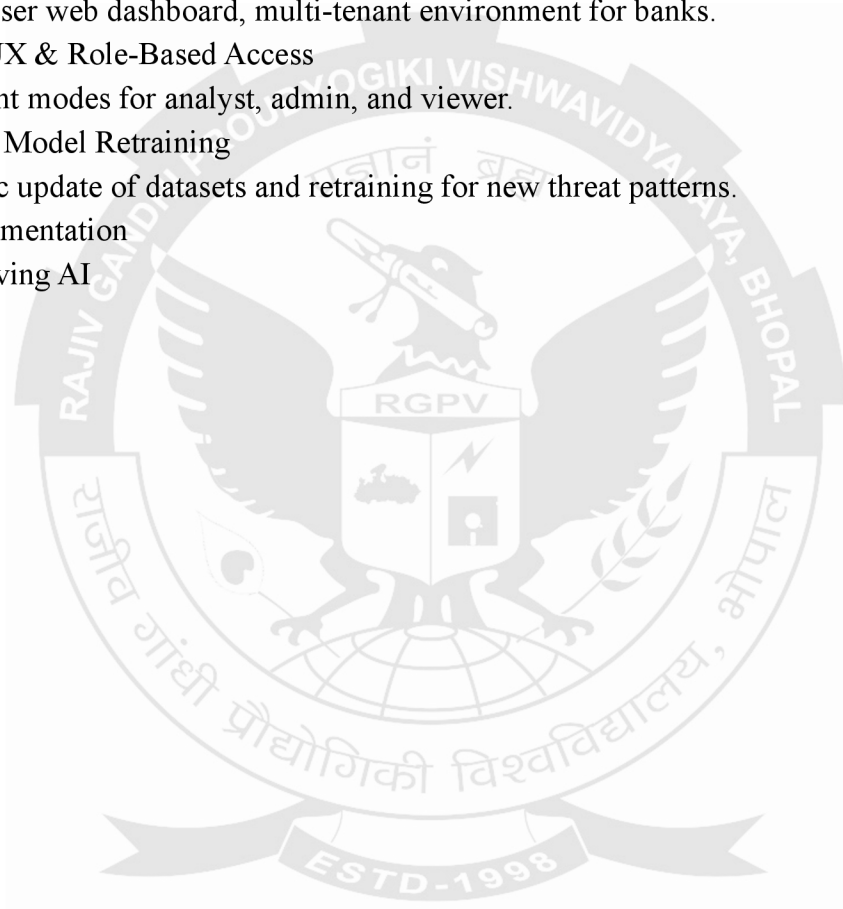
1. Bank Security Operations
 - Validate whether any externally shared APK claiming to be the bank's app is genuine.
2. Government Cybersecurity Units
 - Check APKs collected during fraud investigation.
3. App Store / Third-Party Market Moderation (Future)
 - Integrate into submission pipeline to screen suspicious apps.
4. End-User Protection
 - Give users a tool to check “Is this banking APK fake?” before installing.
5. Academic & Research Use
 - Study and experiment with ML for Android security and fraud detection.



CHAPTER 5: FUTURE SCOPE

Planned enhancements:

1. VirusTotal / MalwareBazar Integration
 - Match file hashes with known malware samples.
2. Dynamic Analysis Engine
 - Run APK in a sandbox emulator, analyze runtime behavior, API calls, network endpoints.
3. Deep Learning Models
 - CNN/RNN/Transformer architectures on DEX opcode sequences.
4. Cloud Dashboard
 - Multi-user web dashboard, multi-tenant environment for banks.
5. Improved UX & Role-Based Access
 - Different modes for analyst, admin, and viewer.
6. Continuous Model Retraining
 - Periodic update of datasets and retraining for new threat patterns.
7. GAN Implementation
8. Self Improving AI



CHAPTER 6: CONCLUSION

Prahari demonstrates how machine learning and static analysis can be combined to build a practical security tool for detecting fake and malicious banking applications. By focusing specifically on banking APKs, it achieves better relevance than generic malware detectors.

Key achievements:

- End-to-end system: from APK upload → analysis → ML prediction → risk report
- Domain-guided synthetic datasets for packages, permissions, and certificates
- Realistic classification performance with clear explainability (flags & scores)
- Simple web-based UI suitable for real-world usage

With further enhancements like integration with threat intelligence APIs and dynamic analysis, Prahari can evolve into a powerful production-grade fraud detection component for mobile banking ecosystems.



BIBLIOGRAPHY

- [1] A. Desnos and G. Gueguen, “Androguard: A Full Python Tool to Play with Android Files — Androguard Documentation Project, 2011.”
- [2] Google Developers, “Android Security & Permissions — Android Developer Documentation, Google LLC, USA.”
- [3] A. Feizollah, N. B. Anuar and R. Salleh, “A Systematic Review on Android Applications Security — IEEE Communications Surveys & Tutorials, Vol. 17, No. 2, 2015.”
- [4] Google Developers, “Android Debug Bridge (ADB) — Official Android Tools Documentation, Google LLC, USA.”
- [5] Jared Rummler, “APKParser: Static APK Parsing Library — GitHub Open-Source Repository.”
- [6] Saurabh Shahane, “Android Permission Dataset — Kaggle Open Data Repository, 2021.”
- [7] Mohit (Abuse.ch), “Malicious Banking APK Collection — MalwareBazaar Threat Intelligence Platform, Abuse.ch.”
- [8] Researcher (Anonymous), “Real-world Banking APK Data (Self-Collected) — Unpublished Dataset for Pattern Study, 2024.”
- [9] P. Kumar and A. Sharma, “Android Malware Detection Through ML-Based Analysis of APK Permissions — IARJSET, Vol. 11, No. 6, 2024.”
- [10] A. Patel, R. Mahajan and D. Rathod, “SecuDroid: Android Malware Detection Using ML Classifier on Static Features — IEEE Xplore Digital Library, 2024.”
- [11] S. Techcrush, “How to Render HTML File in Flask — Medium Technical Article, 2019.”